Практическая работа №3

Подключение Flask и первый маршрут (Hello World)

Герасимов Кирилл Андреевич 3-ИСП44-11

Ссылка на репозиторий: https://github.com/gerasimovkirill022-jpg/prak

Кратко о работе

Цель: Установить веб-фреймворк Flask, создать первое веб-приложение с маршрутами, запустить локальный сервер и открыть страницы в браузере.

Технологии: Python 3; Flask 3.x; протокол HTTP (метод GET, коды 200 и 404); режим отладки debug=True.

Реализация: Приложение создаётся строкой app = Flask(__name__). Декоратор @app.route() связывает URL с функцией-обработчиком. Сервер запускается командой python app.py и слушает адрес http://127.0.0.1:5000. Режим debug=True включает автоматическую перезагрузку при изменении кода. На несуществующий адрес Flask возвращает ответ 404 Not Found.

Основные модули:

- `src/app.py` — Flask-приложение с двумя маршрутами: / (главная страница, возвращает HTML-строку) и /about (страница «О проекте»).
- `src/templates/` — папка для шаблонов (будет использована в работе №4).
- `src/init_db.py, check_db.py, crud_console.py` — файлы из предыдущих работ.

Ход работы

Шаг 2. Структура проекта и Шаг 3. Первый маршрут

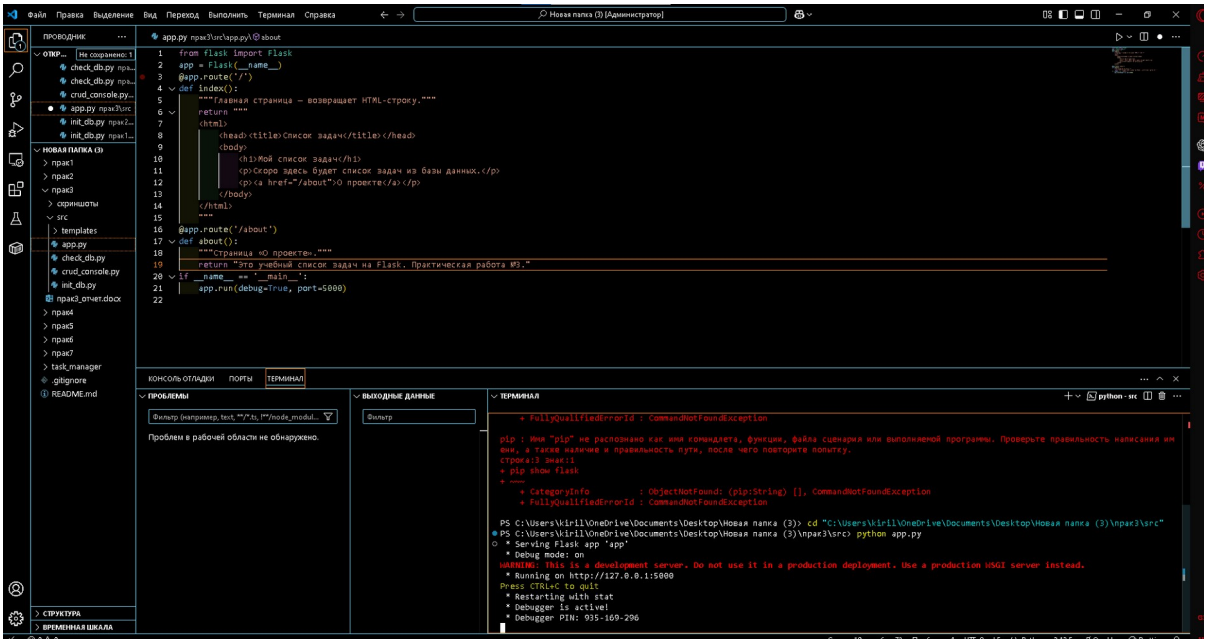


Рисунок 1 — папка templates и код app.py с маршрутами / и /about

Шаг 4. Запуск сервера

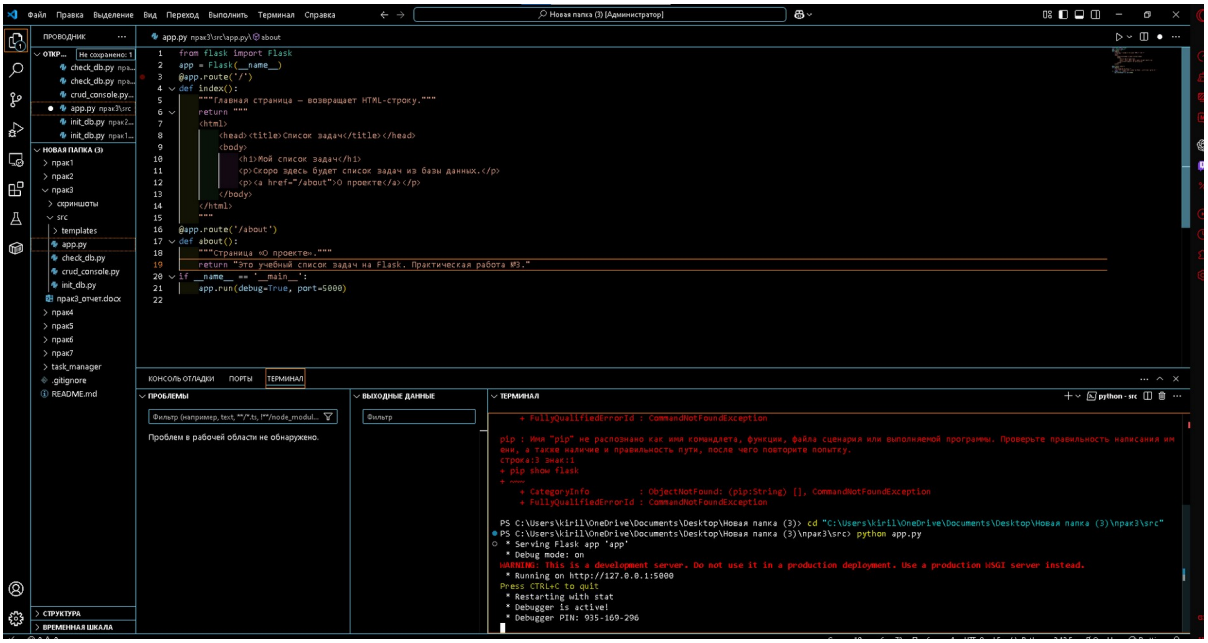


Рисунок 2 — запуск python app.py: сервер работает на http://127.0.0.1:5000, режим отладки включён

Шаг 5. Открытие в браузере

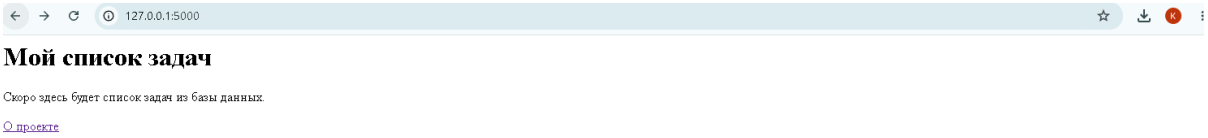


Рисунок 3 — главная страница: HTML с заголовком «Мой список задач» и ссылкой «О проекте»

Шаг 6. Проверка автоперезагрузки

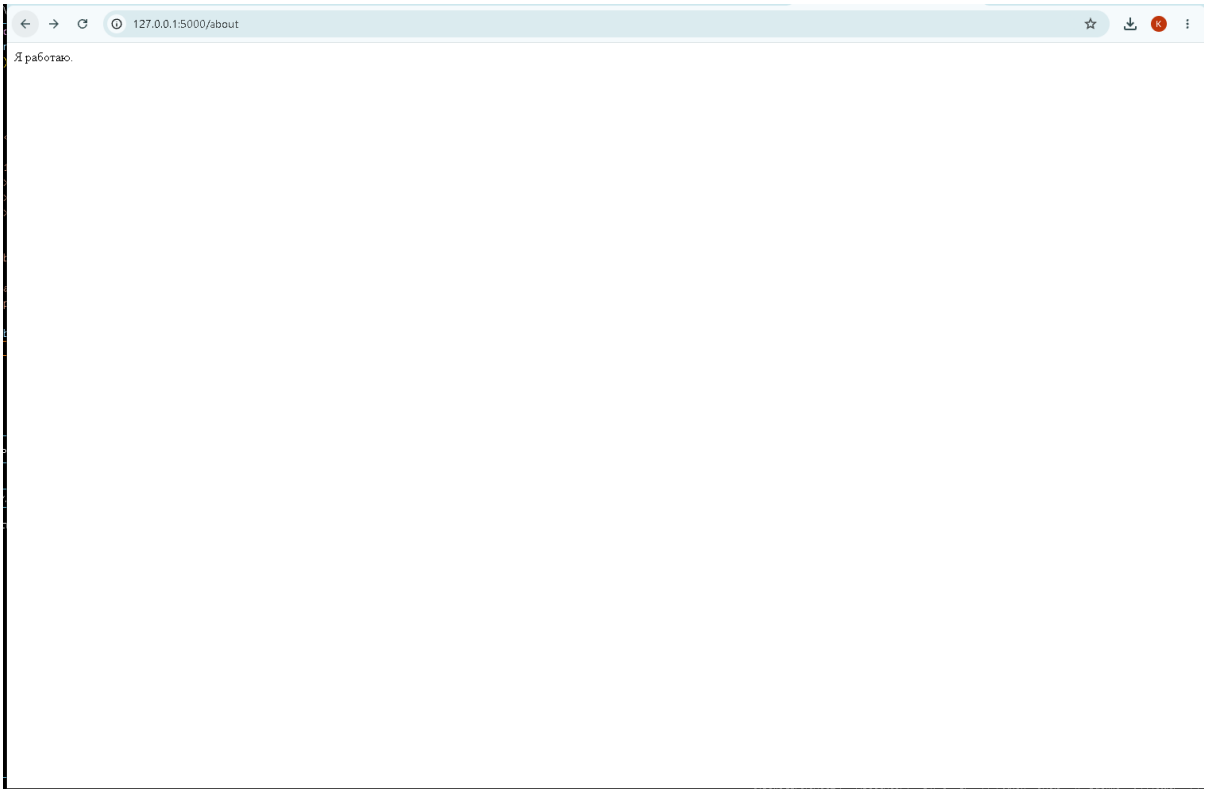


Рисунок 4 — после изменения текста в app.py и сохранения сервер перезагрузился сам — на странице /about новый текст

Шаг 7. Второй маршрут — добавление страницы

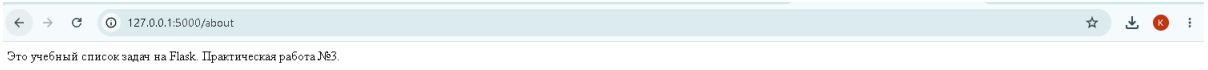


Рисунок 5 — страница /about

Шаг 8. Проверка обработки ошибок 404

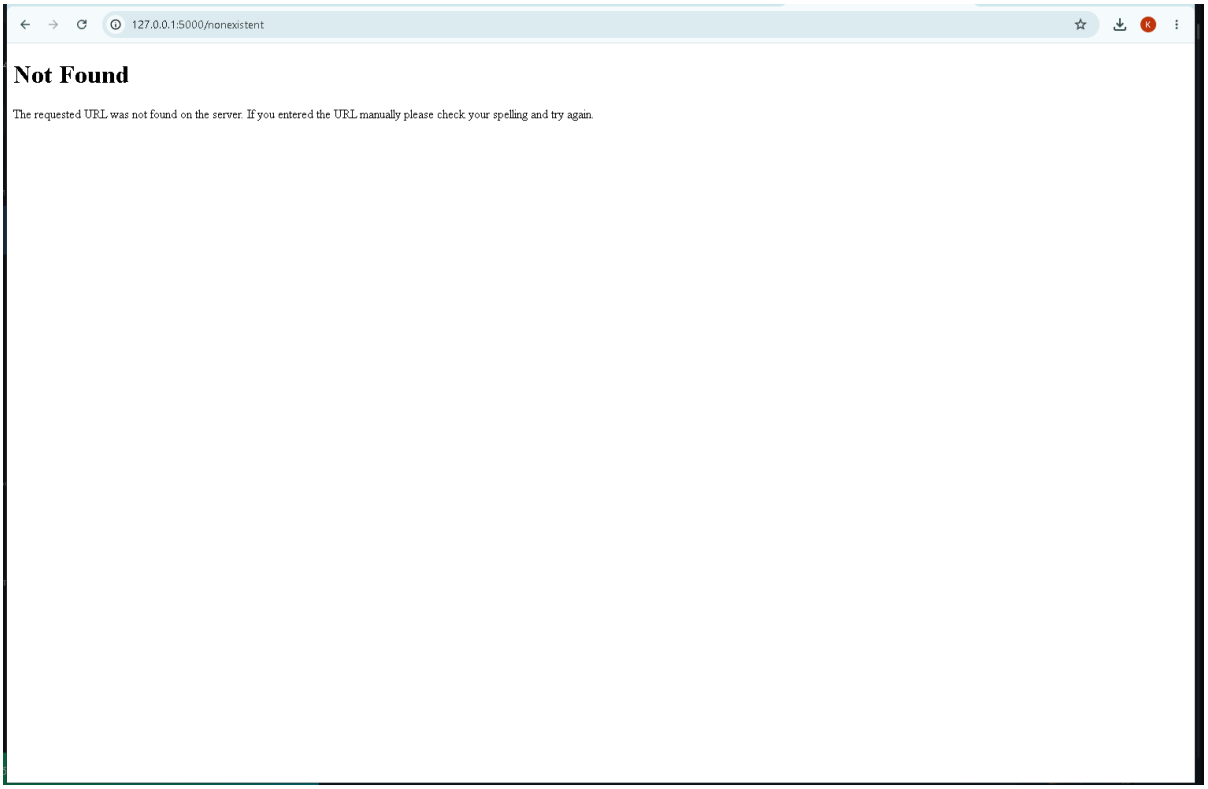


Рисунок 6 — ответ 404 Not Found на несуществующий адрес /nonexistent

Примеры запросов и ответов

```
GET /             → 200 OK   (страница «Мой список задач»)
GET /about        → 200 OK   (текст «Это учебный список задач на Flask…»)
GET /nonexistent  → 404 Not Found
```

Ответы на контрольные вопросы

1. **Что делает декоратор @app.route('/')?**

Регистрирует функцию как обработчик URL «/»: когда браузер запрашивает этот адрес, Flask вызывает эту функцию и отправляет её результат в ответ.

2. **Чем GET отличается от POST? Какой метод по умолчанию?**

GET используется для получения страниц, данные передаются в URL. POST — для отправки данных на сервер, данные в теле запроса. По умолчанию маршрут Flask принимает только GET.

3. **Зачем нужен debug=True и почему его нельзя использовать на реальном сервере?**

Он даёт автоперезагрузку и подробные ошибки в браузере. На реальном сервере это опасно: отладчик раскрывает код и позволяет выполнять команды на сервере.

4. **Что означает адрес 127.0.0.1:5000?**

127.0.0.1 — localhost, собственный компьютер; 5000 — порт, на котором сервер принимает запросы.

5. **Почему при открытии /nonexistent мы получаем 404, а не ошибку Python?**

Flask не нашёл маршрут для этого URL и сам вернул стандартный HTTP-ответ 404 Not Found — код приложения при этом не выполнялся.

6. **Почему папка для HTML-шаблонов должна называться templates?**

Это имя по умолчанию, в котором Flask ищет шаблоны рядом с файлом приложения. При другом имени будет ошибка TemplateNotFound (если не указать папку явно параметром template_folder).

7. **Что произойдёт, если запустить два сервера Flask на одном порту?**

Второй сервер не запустится — появится ошибка «Address already in use» (порт занят).

Вывод

Установлен Flask, создано веб-приложение с двумя маршрутами, проверены работа сервера, переход по ссылке, ответ 404 и автоматическая перезагрузка в режиме отладки.